

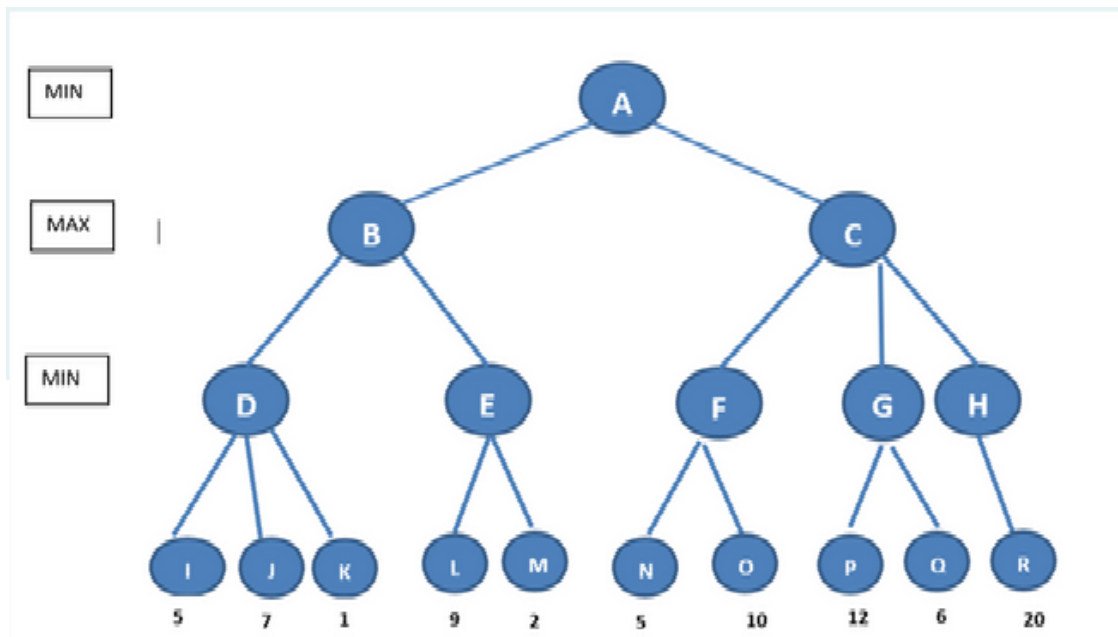
lab8

April 23, 2026

By: Rafay Siddiqui
Student No: 24K-0009
Section: BAI-4A

1 Q1

Apply Both MinMax and Alpha-beta Pruning on the below algorithm.



```
[ ]: import math

tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G', 'H'],
    'D': ['I', 'J', 'K'],
    'E': ['L', 'M'],
    'F': ['N', 'O'],
    'G': ['P', 'Q'],
    'H': ['R'],
```

```

# Leaf node values
'I': 5, 'J': 7, 'K': 1,
'L': 9, 'M': 2,
'N': 5, 'O': 10,
'P': 12, 'Q': 6,
'R': 20
}

def minimax(node, is_maximizing):
    """
    Standard Minimax algorithm without pruning.
    """
    # Base case: if the node is a leaf (its value is an integer), return the
    ↪ value
    if isinstance(tree[node], int):
        return tree[node]

    if is_maximizing:
        best_val = -math.inf
        for child in tree[node]:
            value = minimax(child, False)
            best_val = max(best_val, value)
        return best_val
    else:
        best_val = math.inf
        for child in tree[node]:
            value = minimax(child, True)
            best_val = min(best_val, value)
        return best_val

def alpha_beta(node, alpha, beta, is_maximizing):
    """
    Minimax algorithm optimized with Alpha-Beta pruning.
    """
    # Base case: if the node is a leaf, return the value
    if isinstance(tree[node], int):
        return tree[node]

    if is_maximizing:
        best_val = -math.inf
        for child in tree[node]:
            value = alpha_beta(child, alpha, beta, False)
            best_val = max(best_val, value)
            alpha = max(alpha, best_val)

            # Pruning condition
            if beta <= alpha:

```

```

        print(f" Pruning branch at MAX node '{node}' after evaluating_
↳child '{child}'")
        break
    return best_val
else:
    best_val = math.inf
    for child in tree[node]:
        value = alpha_beta(child, alpha, beta, True)
        best_val = min(best_val, value)
        beta = min(beta, best_val)

        # Pruning condition
        if beta <= alpha:
            print(f" Pruning branch at MIN node '{node}' after evaluating_
↳child '{child}'")
            break
    return best_val

# --- Execution ---

print("--- Running Minimax ---")
minimax_result = minimax('A', is_maximizing=False)
print(f"Optimal value using Minimax: {minimax_result}\n")

print("--- Running Alpha-Beta Pruning ---")
alpha_start = -math.inf
beta_start = math.inf
alphabeta_result = alpha_beta('A', alpha_start, beta_start, is_maximizing=False)
print(f"Optimal value using Alpha-Beta Pruning: {alphabeta_result}")

```

--- Running Minimax ---

Optimal value using Minimax: 2

--- Running Alpha-Beta Pruning ---

Pruning branch at MAX node 'C' after evaluating child 'F'

Optimal value using Alpha-Beta Pruning: 2

2 Q3

Write a Python function `alpha_beta_defense(tree)` to determine the optimal defense score for a financial AI (MAX player) facing an adversary (MIN player) capable of launching a Cyber Attack, Signal Jamming, or a Market Crash. The game tree is provided as a 2D list of terminal defense scores, such as `tree = [[10, 5, 2], [8, 4, 3], [20, 15, 9]]`, where each sublist represents the possible outcomes of a specific attack vector. Your code must implement the standard Minimax algorithm with Alpha-Beta pruning to return the final root value, but crucially, it must first include logic to dynamically sort the three main attack branches to guarantee the absolute maximum number of leaf nodes are pruned during the search.

```

[3]: import math

def alpha_beta_defense(tree):
    optimized_tree = [sorted(branch) for branch in tree]
    optimized_tree.sort(key=lambda x: min(x), reverse=True)

    print(f"Original tree: {tree}")
    print(f"Optimized tree for max pruning: {optimized_tree}\n")

    alpha = -math.inf
    beta = math.inf
    best_defense_score = -math.inf

    print("Beginning Evaluation...")

    for i, branch in enumerate(optimized_tree):
        min_eval = math.inf
        for j, leaf_val in enumerate(branch):
            min_eval = min(min_eval, leaf_val)
            beta = min(beta, min_eval)
            print(f"    Evaluating branch {i}, leaf {j}: value {leaf_val}")

            if beta <= alpha:
                pruned_count = len(branch) - (j + 1)
                print(f"        [!] Cutoff triggered! Alpha ({alpha}) >= Beta_
↳({beta}). Pruned {pruned_count} remaining leaves in branch {i}.")
                break

            best_defense_score = max(best_defense_score, min_eval)
            alpha = max(alpha, best_defense_score)

        beta = math.inf
        print(f"    -> Branch {i} evaluation complete. Current Best Defense_
↳(Alpha): {alpha}\n")

    return best_defense_score

game_tree = [[10, 5, 2], [8, 4, 3], [20, 15, 9]]

optimal_score = alpha_beta_defense(game_tree)
print(f"=== Final Optimal Defense Score: {optimal_score} ===")

```

Original tree: [[10, 5, 2], [8, 4, 3], [20, 15, 9]]

Optimized tree for max pruning: [[9, 15, 20], [3, 4, 8], [2, 5, 10]]

Beginning Evaluation...

```
Evaluating branch 0, leaf 0: value 9
Evaluating branch 0, leaf 1: value 15
Evaluating branch 0, leaf 2: value 20
-> Branch 0 evaluation complete. Current Best Defense (Alpha): 9

Evaluating branch 1, leaf 0: value 3
  [!] Cutoff triggered! Alpha (9) >= Beta (3). Pruned 2 remaining leaves in
branch 1.
-> Branch 1 evaluation complete. Current Best Defense (Alpha): 9

Evaluating branch 2, leaf 0: value 2
  [!] Cutoff triggered! Alpha (9) >= Beta (2). Pruned 2 remaining leaves in
branch 2.
-> Branch 2 evaluation complete. Current Best Defense (Alpha): 9

=== Final Optimal Defense Score: 9 ===
```

3 Q4

Implement Super Tic Tac To using MinMax and AlphaBeta Algorithm.